Retrace Repository Conventions Specification

Version: 1.0

Date: 6 November 2026

Status: Approved repository-conventions baseline

Applies to: the Retrace monorepo, its services, web application, shared libraries, contracts, infrastructure, CI/CD, and coding-agent work

1. Purpose, authority, and status labels

This specification turns settled repository working decisions into practical rules for implementing Retrace. It preserves the product and architectural decisions already made in the PRD, System Design, Database Design, Project Context, AI Pipeline Specification v0.2, and API & Event Contracts Specification v1.0. It does not replace those documents or create unresolved services, API fields, database columns, provider behavior, thresholds, or agent assignments.

Use the following labels throughout this document:

Label	Meaning
Confirmed project decision	An explicit decision settled in the project conversation. It is mandatory unless a later approved decision supersedes it.
Established architectural constraint	A binding design already established by a source specification. Repository work must preserve it.
Standard convention	A compatibility-oriented repository rule chosen to make the confirmed decisions operable. It must not change product behavior or service ownership.
Deferred item	An intentionally unresolved choice. Do not guess; resolve it in the applicable implementation plan, ADR, or contract work before depending on it.

When sources disagree, use the following reconciliation order for repository work: explicit confirmed decisions in the current repository/API contract documents; the AI Pipeline Specification for AI behavior; the API & Event Contracts Specification for wire contracts; the System Design for architecture and service boundaries; the Database Design for persistence ownership and existing columns; and the PRD for product intent. The Project Context is the living index and should be corrected when it conflicts with the more detailed source.

2. Confirmed decisions register

Twenty-four repository decisions are locked. Q24 is deferred by the project owner and is not a repository-wide policy.

ID	Area	Confirmed decision
Q1	Python dependencies	Use `uv`.
Q2	Frontend dependencies	Use `pnpm`.
Q3	Python version	Target Python 3.12.
Q4	Backend structure	Use a hybrid feature-based structure with shared infrastructure.
Q5	Frontend structure	Use a hybrid feature-based structure with shared modules.
Q6	Frontend language	Use TypeScript.
Q7	Python linting/formatting	Use Ruff.
Q8	Python testing	Use pytest and pytest-django.
Q9	Frontend linting/formatting	Use ESLint and Prettier.
Q10	Frontend testing	Use Vitest, React Testing Library, and Playwright.
Q11	Git workflow	Use feature branches and pull requests.
Q12	Commit messages	Use Conventional Commits.
Q13	Node versioning	Use an LTS Node.js release pinned by `.nvmrc`.

ID	Area	Confirmed decision
Q14	Python workspace	Use a hybrid uv workspace.
Q15	Secrets	Use local `.env` files and a production secret manager.
Q16	Local environment	Use root Docker Compose with override support.
Q17	PR protection	Require CI and one approval.
Q18	Delivery	Automatically deploy staging; manually approve production.
Q19	Migrations	Execute through the controlled CI/CD migration workflow.
Q20	Coverage	Introduce enforcement gradually.
Q21	API client pattern	Generate types and write API functions by hand.
Q22	Generated output	Use a dedicated generation command and commit generated output.
Q23	Documentation	Keep central cross-service documentation and service-level documentation.
Q25	Naming	Follow language-standard naming conventions.

3. Architectural constraints that repository work must preserve

Retrace is a monorepo for independently deployable services that may later be split into separate repositories. V1 consists of Gateway, Core, Ingestion, AI, Notifications, and the web application. Realtime is a later-phase service and must not become a v1 dependency.

Component	Established responsibility	Framework/runtime constraint
Gateway	TLS, routing, JWT verification, CORS, rate limiting, request IDs; no business data	Traefik
Core	Identity, organizations, tenancy registry, incidents, postmortems, comments, approvals, action items, analytics, exports, audit log, job status	Django + Django REST Framework
Ingestion	Integrations, allowlists, catalog, evidence, timeline, PagerDuty webhooks	Django + Django REST Framework + Celery
AI	Model settings, draft runs, regeneration, blameless checks, inline suggestions, usage limits	Django Ninja + Celery
Notifications	In-app notifications, preferences, reminder and email-delivery records	Django + Django REST Framework + Celery beat
Web	User interface	React + Vite + TypeScript + Tailwind CSS + shadcn/ui + TanStack Query + TipTap

The following are non-negotiable repository boundaries:

- A service must never import another service's application code or access its database schema. Cross-service work uses approved internal REST APIs or events.
- Common reusable implementation code belongs only in `libs/`, is packaged/versioned deliberately, and must not conceal service-specific business ownership.
- Public and internal API/event definitions belong centrally in `contracts/`; producers must not silently change a consumer-used schema.
- Each service has its own schema, role, migration history, outbox, inbox, job queues, and ownership of migrations for that schema.
- Tenant routing, authorization, and isolation must hold for requests, jobs, caches, events, tests, and direct database access. No repository shortcut may bypass this boundary.

4. Required monorepo structure

The repository shall follow this target layout. Names below are authoritative for top-level placement; a service may add only files necessary for its framework and documented responsibility.

```
retrace/
  services/
    core/
    ingestion/
```

```
    ai/
    notifications/
    realtime/                 # later phase; no v1 runtime dependency
  web/
  gateway/
  libs/
    retrace_auth/
    retrace_tenancy/
    retrace_events/
    retrace_common/
  contracts/
    common/
    core/
    ingestion/
    ai/
    notifications/
    events/
  infra/
  docs/
    decisions/
  .github/
    workflows/
  compose.yaml
  compose.override.yaml.example
  .env.example
  .nvmrc
  pyproject.toml
  uv.lock
  pnpm-workspace.yaml
  pnpm-lock.yaml
```

4.1 Service layout

Each active Python service must contain its Django project/application structure, `api/`, `services/`, `tasks/` where it owns asynchronous work, `events/`, `tests/`, a service `Dockerfile`, and its OpenAPI publication/configuration. Place feature code close to the feature it serves; retain framework adapters and broadly reusable infrastructure in the appropriate shared module.

Feature modules may own their models, serializers/schemas, handlers, business services, and focused tests. View/controller code remains thin: request validation, authorization entry, and delegation. Business rules that protect lifecycle, tenancy, state transitions, or cross-resource constraints must not be duplicated in views, tasks, and event handlers.

4.2 Frontend layout

The web application uses a hybrid feature-based structure:

```
web/src/
  app/             # application bootstrap, routing, providers
  features/        # incident, postmortem, evidence, action-item, etc.
  components/      # shared presentational components
  api/             # handwritten query/mutation functions and client wiring
  generated/       # committed generated API types; never hand-edit
  hooks/           # reusable cross-feature hooks
  lib/             # framework-neutral helpers
  styles/
  test/
```

Do not create a generic shared module for one feature's private logic. Promote code only after a genuine second consumer exists or a cross-cutting platform concern requires it.

4.3 Contracts, infrastructure, and documentation

`contracts/` contains common OpenAPI components, a machine-readable OpenAPI 3.x root and fragments per public service, internal schemas where finalized, and versioned event payload schemas grouped by producer. `infra/` contains Docker Compose assets, Azure infrastructure files, and the migration-runner assets. Root `docs/` contains cross-service documentation; `docs/decisions/` contains ADRs. Each service README records its responsibility, local commands, environment variables, health endpoint, and contract/migration ownership.

5. Python tooling and workspace management

5.1 Runtime and dependencies

- **Confirmed project decision:** Python 3.12 is the supported implementation version.
- **Confirmed project decision:** `uv` is the only Python dependency and environment workflow for new repository automation.
- **Confirmed project decision:** use a hybrid `uv` workspace: root configuration centralizes shared tooling and workspace membership; each independently deployable service and reusable library has its own package metadata and declared dependencies.
- Commit `uv.lock`. Do not add an unpinned second resolver, a service-local lockfile, or undeclared runtime dependencies.
- Keep production dependencies minimal and service-scoped. Development/test tooling belongs in the appropriate dependency group rather than production images.

Recommended command names may be exposed through documented task targets, but command wrappers must call `uv` rather than duplicating environment setup. New package creation, dependency upgrades, and lockfile refreshes must be reviewed as source changes.

5.2 Python quality rules

- **Confirmed project decision:** Ruff performs Python formatting and linting. A single root configuration establishes the baseline; narrow service-specific exceptions require a documented reason.
- **Confirmed project decision:** pytest and pytest-django are the Python test baseline.
- Run formatting, linting, unit tests, and relevant service tests locally before opening a PR. CI is the final authority.
- Tests must cover normal, validation, authorization, cross-tenant, idempotency/concurrency, retry, and failure behavior where the changed code participates in those paths.
- Keep test fixtures tenant-aware. A test that does not explicitly set tenant context must fail safely where tenant data access is possible.

6. Frontend tooling and workspace management

6.1 Runtime and dependencies

- **Confirmed project decision:** TypeScript is mandatory for `web/` source and tests.
- **Confirmed project decision:** use `pnpm` for frontend dependencies and workspace management.
- **Confirmed project decision:** Node.js must be an LTS release pinned in `.nvmrc`. CI and local development use that pinned version; updates are intentional repository changes.
- Commit `pnpm-lock.yaml` and use frozen/immutable lockfile installation in CI.
- Declare workspace packages in `pnpm-workspace.yaml`; do not mix npm, Yarn, or untracked global frontend dependencies into the repository workflow.

6.2 Frontend quality rules

- **Confirmed project decision:** ESLint and Prettier are the frontend linting and formatting tools. Their configs are shared at the web/workspace level.
- **Confirmed project decision:** Vitest, React Testing Library, and Playwright form the frontend test stack.
- Use React Testing Library for behavior a user can observe; use Playwright for critical cross-browser end-to-end flows. Avoid snapshot-only tests for business workflow behavior.
- Keep remote data access in typed API/query layers. Components must not hand-parse untyped response data or repeat transport rules.

7. Dependency, lockfile, and workspace rules

1. Every runtime dependency must be declared in the owning package and represented in the authoritative lockfile.
2. Do not import an implementation across service boundaries to avoid adding a contract. Add or update an API/event contract instead.

3. Shared Python packages in `libs/` are installed through the workspace during monorepo development and must remain publishable/splittable in principle.
4. Generated code is not an independently maintained dependency. Its generator, source contract, and checked-in output must move together.
5. Pin production container base images and keep dependency updates narrow, reviewable, and covered by CI.
6. Supply-chain checks, image scanning, and dependency alerts remain part of CI as established by the System Design.

8. Formatting, linting, testing, and coverage

Quality checks are required repository gates. The root task documentation should expose predictable commands for format-check, lint, type-check, unit test, contract test, integration test, and end-to-end test without changing the settled tool choices.

Scope	Required baseline
Python	Ruff format/lint; pytest; pytest-django where Django is used
Frontend	ESLint; Prettier; TypeScript checks; Vitest; React Testing Library; Playwright
API/event contracts	Schema validation, compatibility/fixture tests, producer-consumer coverage
Tenant isolation	Two-tenant route/job tests and direct service-role/schema tests where applicable
AI	Evidence-only, citation, contradiction, timestamp-uncertainty, and no-human-overwrite tests from the AI Pipeline Specification

Confirmed project decision: coverage enforcement is gradual. CI must collect and publish coverage from the beginning, but no repository-wide percentage threshold is implied until it is approved. New code should not reduce the confidence of an affected service; add focused coverage when fixing defects or touching high-risk logic.

9. Git, commits, pull requests, and review

9.1 Branches and commits

Confirmed project decision: work occurs on focused feature branches and is integrated through pull requests. Keep a branch limited to one coherent change set; avoid bundling toolchain upgrades, generated output churn, unrelated refactors, and behavior changes unless they are inseparable.

Confirmed project decision: use Conventional Commits:

```
type(scope): imperative summary
```

Use an appropriate type such as `feat`, `fix`, `docs`, `test`, `refactor`, `build`, `ci`, `chore`, or `perf`. Scopes identify the affected service, `web`, `contracts`, `infra`, or `libs`. A breaking change must be explicit in the commit/PR and accompanied by a contract and migration plan.

9.2 Pull-request requirements

Confirmed project decision: protected branches require all required CI checks and at least one approval before merge. A PR must include:

- a concise problem and implementation summary;
- the affected services, contracts, migrations, generated files, and configuration changes;
- test evidence and any intentionally unrun checks;
- a rollback or compatibility note for schema, event, migration, or deployment changes;
- linked ADR/open item where a decision is being resolved.

Never merge a producer contract change without regenerated consumer types/fixtures and compatibility evidence. Never combine an application migration with an uncontrolled production database action.

10. Naming and repository language

Confirmed project decision: Retrace follows language-standard naming rather than enforcing one cross-language spelling style.

Element	Python	TypeScript/React
Modules, directories	`snake_case` where Python tooling expects it	`kebab-case` for ordinary directories/files; components use `PascalCase.tsx`
Variables/functions	`snake_case`	`camelCase`
Classes/types/components	`PascalCase`	`PascalCase`
Constants	`UPPER_SNAKE_CASE`	`UPPER_SNAKE_CASE`
Database tables/columns	`snake_case`; plural tables; foreign-key columns end in `_id`	Not applicable

Database conventions are established constraints: application-generated UUIDv7 primary keys; `timestamptz` stored in UTC; text plus check constraints rather than database enum types; and no foreign keys across service schemas. Do not rename or re-style existing contract names merely to fit a frontend preference.

11. Secrets and configuration

11.1 Local development

Confirmed project decision: local secrets and configuration use `.env` files. Commit `.env.example` files containing variable names, safe defaults, and setup guidance only. Never commit values, token exports, provider credentials, private keys, production connection strings, or copied secret-manager material.

Configuration must be explicit, typed, and validated at service startup. Environment variables are for deployment-specific configuration; business defaults and behavior that require product agreement must not be silently hidden in environment settings. Logs, test snapshots, error responses, and generated artifacts must not reveal secrets or unredacted evidence.

11.2 Deployed environments

Established architectural constraint: production secrets use Azure Key Vault. Integration tokens and model keys remain tenant-scoped ciphertext protected by a per-tenant data key wrapped by the Key Vault master key. Services use their scoped managed identity/production identity and minimum permissions. The repository must contain references, names, and templates, never decrypted values.

12. Docker Compose local development

Confirmed project decision: local development uses root Docker Compose with override support. The root Compose definition is the authoritative local topology; `compose.override.yaml` remains local/uncommitted, while `compose.override.yaml.example` documents supported overrides.

The Compose environment may run the gateway, active services, workers, PostgreSQL/control and local tenant-development resources, Redis, and needed local storage substitutes. It must preserve service boundaries: use service-specific containers, queues, and configuration rather than replacing cross-service behavior with in-process imports.

Local convenience does not authorize a weaker security model in code. Tenant context, service authentication paths, schema boundaries, migration ownership, outbox/inbox behavior, and redaction protections must be testable in local development. Never require a production Key Vault secret to start the documented local environment.

13. CI/CD, images, environments, and migration workflow

13.1 CI/CD

Confirmed project decision: staging deploys automatically after the required CI and merge conditions are met; production deployment requires a manual approval gate. GitHub Actions is the established delivery system and uses path-aware workflows so a changed service, its shared library, contracts, or infrastructure runs the checks that can be affected.

CI must at minimum validate formatting/linting, type checks, unit/API tests, contract schemas and fixtures, relevant integration tests, generated API output freshness, dependency/lockfile consistency, image build checks, and the changed service's migration packaging. Build/deploy workflows must not turn an unapproved open implementation item into an infrastructure default.

13.2 Migrations

Confirmed project decision: migrations execute only through the controlled CI/CD migration workflow. Local commands may create and validate migrations, but production migration execution is not a developer workstation action.

Each service owns migrations for its own schema. The migration runner is a separate infrastructure container job that reads the tenant registry, runs each service's migrations with that service's role, records the per-tenant/per-service version, limits concurrency, reports failures, and continues safely for unaffected tenants. New-tenant provisioning applies all required service migrations idempotently before activation.

Migration PRs must include forward behavior, compatibility with running code, tenant rollout behavior, validation, and rollback/mitigation notes. Do not add cross-schema foreign keys, direct schema access, or unreviewed destructive operations. Contract-required persistence changes, including evidence timestamps and soft-delete lifecycle fields, require an approved migration before API code claims they exist.

14. API contracts and generated frontend types

14.1 Contract-first workflow

The central `contracts/` directory is the source of truth for shared API/event schemas. Public REST APIs use `/api/v1`; tenant routes use `/api/v1/orgs/{org_id}/` except established authentication and organization-creation exceptions. API rules, event envelopes, pagination, errors, idempotency, ETags, evidence provenance, and open contract items remain governed by API & Event Contracts Specification v1.0.

For every contract-affecting change:

1. update the owning OpenAPI fragment/root or versioned event schema in `contracts/`;
2. update the producing service publication/implementation and schema tests;
3. regenerate committed frontend types;
4. update handwritten API functions, fixtures, and consuming tests;
5. run contract compatibility and cross-tenant tests in CI.

14.2 Type generation

Confirmed project decisions: API types are generated, API functions remain handwritten, generation is performed by a dedicated command such as `pnpm generate:api`, and generated output is committed. Generated files live under `web/src/generated/`, carry a generated-file notice, and must never be hand-edited. CI fails when the command would change committed generated output.

Generated types must reflect the approved OpenAPI source. They do not authorize clients to invent endpoints, internal route details, numeric limits, missing fields, or provider semantics that remain open.

15. Documentation and architecture decisions

Confirmed project decision: Retrace uses hybrid documentation.

- Root `docs/` holds architecture, local setup, operations, conventions, cross-service workflows, and integration guidance.
- `docs/decisions/` holds ADRs for durable architectural decisions and changes to prior decisions.
- Each service owns a README for service-specific setup, commands, environment variables, health checks, migrations, APIs/events, and operational concerns.
- Contract behavior is documented centrally under `contracts/` and the API/Event Contracts specification; implementation READMEs link rather than fork authoritative definitions.

An ADR is required before materially changing a confirmed framework, infrastructure, tenancy, communication, service boundary, or repository-tooling choice. Each ADR states context, decision, consequences, affected specifications, migration/compatibility impact, and status. Do not use ADRs to silently decide an item explicitly marked deferred.

16. Coding-agent instructions

Every implementation task for Cursor, Antigravity, or Codex must state the allowed paths, owning service(s), required contracts, tests, migration impact, and expected handoff. Before changing code or contracts, an agent must read the PRD, System Design, Database Design, Project Context, AI Pipeline Specification v0.2, API & Event Contracts Specification v1.0, and this specification as applicable to the task.

Agents must:

1. preserve the service/data ownership boundary and use contracts for cross-service integration;
2. search existing contracts, migrations, and tests before adding a route, event, column, or shared abstraction;
3. update contract schemas before implementation when a public or cross-service interface changes;
4. keep secrets and tenant data out of logs, fixtures, prompts, and generated output;
5. make migration, generated-file, and documentation effects explicit in the PR;
6. stop and record an open item rather than inventing an unresolved API detail, provider behavior, numerical threshold, database column, or owner;
7. avoid unrelated refactors and preserve user changes in shared files.

17. Deferred decisions and prohibited assumptions

17.1 Q24 - agent task and service ownership

Deferred by explicit project decision. Task allocation, permitted file boundaries, service ownership, dependencies, integration points, review responsibility, and sequencing for Cursor, Antigravity, and Codex will be defined in separate implementation plans. This document creates no permanent agent ownership map and assigns no service to an agent.

17.2 Existing open implementation items

The repository must not silently decide these established open items: exact API request/response fields and filters; webhook public-path wiring; service-token claims and internal route schemas; event payload details and aggregate-version sources; evidence snapshot storage mapping; migration-backed evidence timestamp and soft-delete fields; explicit-save and version-retention mechanics; numeric page sizes, limits, retries, backoff, and dead-letter retention; Azure model/default quota; email provider; custom domain; demo dataset; infrastructure-as-code tool; and connection-pooler deployment details.

Resolve each item in its proper contract, ADR, implementation plan, or source-document update before code depends on it.

18. Implementation checklist

Repository foundation

- Create the required top-level structure without creating a v1 Realtime dependency.
- Add Python 3.12, root uv workspace configuration, service/library package metadata, and `uv.lock`.
- Add LTS `.nvmrc`, `pnpm-workspace.yaml`, TypeScript web configuration, and `pnpm-lock.yaml`.
- Add root Ruff, pytest, ESLint, Prettier, Vitest, and Playwright configurations.
- Add `.env.example`, Compose example overrides, and documented local startup/quality commands.

Service and contract safety

- Create service skeletons with no cross-service imports or direct cross-schema access.
- Establish `libs/` package boundaries for auth, tenancy, events, and common infrastructure only.
- Establish central `contracts/` folders, OpenAPI publication, event-schema validation, fixtures, and ownership checks.
- Implement API generation and verify `pnpm generate:api` leaves no uncommitted changes in CI.
- Add baseline two-tenant isolation, contract, idempotency/concurrency, and event duplicate/stale-order tests.

Delivery and documentation

- Add path-aware GitHub Actions checks, required PR status checks, and one-approval branch protection.
- Add automatic staging deployment and manual production approval gates.
- Add the separate migration-runner job, per-tenant recording, failure reporting, and rollout safeguards.
- Add root documentation, ADR template/location, and service README template.
- Write individual implementation plans before assigning coding-agent task/service ownership.

19. Source baseline

This specification was reconciled against the uploaded Retrace PRD, System Design, Database Design, Project Context, AI Pipeline Specification v0.2, and API & Event Contracts Specification v1.0, plus the twenty-five repository-convention questions settled in the referenced project conversation. It deliberately preserves established architecture and records Q24 as deferred.